4.3.5 *Higher Order Symplectic Methods:* SHAKE *and* RATTLE

The projected symplectic constrained method (4.20)–(4.24) is only first order accurate. We forego providing a detailed proof of this fact, but note that it could be demonstrated using standard methods [164]. Note that (4.20)–(4.24) reduces to the symplectic Euler method in the absence of constraints, and the projection of the momenta would not alter this fact. There are several constraint-preserving, second-order alternatives which generalize the Störmer-Verlet scheme. One of these is the SHAKE method [322]. The original derivation of the SHAKE method began from the position-only, two-step form of the Störmer rule for $\ddot{\boldsymbol{q}} = \boldsymbol{F}(\boldsymbol{q})$

$$\boldsymbol{q}_{n+1} - 2\boldsymbol{q}_n + \boldsymbol{q}_{n-1} = h^2 \boldsymbol{M}^{-1} \boldsymbol{F}(\boldsymbol{q}_n).$$

In the presence of the constraint, we consider the scheme

$$\boldsymbol{q}_{n+1} - 2\boldsymbol{q}_n + \boldsymbol{q}_{n-1} = h^2 \boldsymbol{M}^{-1} \boldsymbol{F}(\boldsymbol{q}_n) - h^2 \boldsymbol{g}'(\boldsymbol{q}_n)^T \lambda_n, \tag{4.25}$$

$$\boldsymbol{g}(\boldsymbol{q}_{n+1}) = 0. \tag{4.26}$$

The symmetry suggests that this will be second order. In order to understand the symplectic property associated to this method, we need to define the updates for both positions and momenta. A natural choice is to consider the phase space formulation of the Störmer-Verlet method for $\dot{\boldsymbol{q}} = \boldsymbol{M}^{-1}\boldsymbol{p}, \dot{\boldsymbol{p}} = \boldsymbol{F}(\boldsymbol{q})$, then replace $\boldsymbol{F}$ by $\boldsymbol{F} - \boldsymbol{g}'(\boldsymbol{q})^T\lambda$, resulting in

$$\boldsymbol{q}_{n+1} = \boldsymbol{q}_n + h\boldsymbol{M}^{-1}\boldsymbol{p}_{n+1/2}, \tag{4.27}$$

$$\boldsymbol{p}_{n+1/2} = \boldsymbol{p}_n + (h/2)\boldsymbol{F}(\boldsymbol{q}_n) - (h/2)\boldsymbol{g}'(\boldsymbol{q}_n)^T \boldsymbol{\lambda}_n, \tag{4.28}$$

$$\boldsymbol{p}_{n+1} = \boldsymbol{p}_{n+1/2} + (h/2)\boldsymbol{F}(\boldsymbol{q}_{n+1}) - (h/2)\boldsymbol{g}'(\boldsymbol{q}_{n+1})^T \lambda_{n+1}. \tag{4.29}$$

It is possible to demonstrate (see exercises) that this method reduces to (4.25)–(4.26) by eliminating the momenta. However (4.27)–(4.29) does not give a map of the cotangent bundle, since $\boldsymbol{g}'(\boldsymbol{q}_{n+1})\boldsymbol{p}_{n+1} = 0$ will not be preserved, even if both the constraint and hidden constraint hold at step n.

A method that automatically preserves both the constraint and hidden constraint is the RATTLE method [12] which consists of the following steps:

$$\boldsymbol{q}_{n+1} = \boldsymbol{q}_n + h\boldsymbol{M}^{-1}\boldsymbol{p}_{n+1/2}, \tag{4.30}$$

$$\boldsymbol{p}_{n+1/2} = \boldsymbol{p}_n + (h/2)\boldsymbol{F}_n - (h/2)\boldsymbol{g}'(\boldsymbol{q}_n)^T \boldsymbol{\lambda}_n, \tag{4.31}$$

$$\boldsymbol{0} = \boldsymbol{g}(\boldsymbol{q}_{n+1}), \tag{4.32}$$

$$\boldsymbol{p}_{n+1} = \boldsymbol{p}_{n+1/2} + (h/2)\boldsymbol{F}_{n+1} - \boldsymbol{g}'(\boldsymbol{q}_{n+1})^T \boldsymbol{\mu}_{n+1}, \tag{4.33}$$

$$\boldsymbol{0} = \boldsymbol{g}'(\boldsymbol{q}_{n+1})\boldsymbol{M}^{-1}\boldsymbol{p}_{n+1}. \tag{4.34}$$

In practice, the implementation of this method is essentially the same as that of the constrained Symplectic Euler method (including the cotangency projection). Moreover, the symplecticness of (4.30)–(4.34) may be demonstrated using a very similar approach to that used to demonstrate the symplectic condition for (4.20)–(4.24).

What is not perhaps immediately obvious is that RATTLE and SHAKE are in fact equivalent methods in a certain, precisely defined sense. The SHAKE method defines a mapping $\mathcal{G}_h^{\mathrm{S}}$ of $\mathcal{M} = \{(\boldsymbol{q},\boldsymbol{p}) | \boldsymbol{g}(\boldsymbol{q}) = 0\}$. That is,starting from $z_0 = (\boldsymbol{q}_0,\boldsymbol{p}_0) \in \mathcal{M}$, it generates a set of points $z_1, z_2, \ldots \in \mathcal{M}$ such that $z_{n+1} = \mathcal{G}_h^{\mathrm{S}}(z_n)$. In a similar way, RATTLE defines a mapping of the lower dimensional cotangent bundle, $\mathcal{G}_h^{\mathrm{R}} : T^*\mathcal{M} \to T^*\mathcal{M}$. Even though these methods are maps defined on sets of different dimensionalities, the iterations generated by them can be related to the following staggered iteration

$$\boldsymbol{q}_{n+1} = \boldsymbol{q}_n + h\boldsymbol{M}^{-1}\boldsymbol{p}_{n+1/2}, \tag{4.35}$$

$$\boldsymbol{p}_{n+1/2} = \boldsymbol{p}_{n-1/2} + h\boldsymbol{F}_n - h\boldsymbol{g}'(\boldsymbol{q}_n)^T\tilde{\boldsymbol{\lambda}}_n, \tag{4.36}$$

$$\boldsymbol{0} = \boldsymbol{g}(\boldsymbol{q}_{n+1}), \tag{4.37}$$

where, for SHAKE, $\tilde{\boldsymbol{\lambda}}_n = \boldsymbol{\lambda}_n$, while, in case of RATTLE, we must define $\tilde{\boldsymbol{\lambda}}_n = (\boldsymbol{\lambda}_n + \boldsymbol{\mu}_n)/2$. The proof of this is contained in one of the exercises. Given an initial condition $(\boldsymbol{q}_0^{\mathrm{S}}, \boldsymbol{p}_0^{\mathrm{S}})$ for the SHAKE discretization, we easily determine $\boldsymbol{\lambda}_0^{\mathrm{S}}$ by taking a step. Then it follows that

$$\boldsymbol{p}_{-1/2} = \boldsymbol{p}_0 - (h/2)\boldsymbol{F}(\boldsymbol{q}_0^{\mathrm{S}}) - (h/2)\boldsymbol{g}'(\boldsymbol{q}_0^{\mathrm{S}})^T\boldsymbol{\lambda}_0^{\mathrm{S}},$$

by solving (4.29). Next we obtain $\boldsymbol{p}_0^{\mathrm{R}}$ by solving (4.33)–(4.34) with $n = -1$. Then the sequences generated using $(\boldsymbol{q}_0^{\mathrm{S}}, \boldsymbol{p}_0^{\mathrm{S}})$ as the initial point for SHAKE and that generated using $(\boldsymbol{q}_0^{\mathrm{S}}, \boldsymbol{p}_0^{\mathrm{R}})$ in RATTLE will have identical positions (the momenta can be made compatible by simply projecting the SHAKE momenta onto the constraints).

Given the above discussion and derivation, the proof of symplecticness of the RATTLE method (4.30)–(4.34) is now a formality. For details refer to [229].

4.3.5.1 Symmetric Composition Methods with Constraints

Recall that the Störmer-Verlet method could be constructed by composing steps using Symplectic Euler and its adjoint method. Using more complicated methods it is possible to build higher order schemes. It seems natural that a similar procedure should be possible in the constrained setting. But what, precisely, is the adjoint method in the case of (4.20)–(4.24)?